Abstract

This report compares three distinct strategies for searching a student registration number: iterative linear search, recursive binary search, and hash-table lookup. The assignment requires analysis for datasets of approximately 1,000, 10,000, 100,000 and 1,000,000 records, including mathematical complexity, best/average/worst cases, recurrence analysis, experimental measurements and the Day-4 requirement of 50,000 searches per hour. A Python benchmark script and measured output files accompany this report. Timings depend on the computer and Python runtime; rerun the supplied script on the submission computer and capture genuine screenshots before final submission.

Problem statement and assumptions

Each student has a unique registration number. The database must determine whether a requested number exists and return its record/index. The four tested dataset sizes follow the assignment. For a fair comparison, the benchmark creates the same unique integer keys, uses 30 successful random searches per algorithm and size, and checks missing-key cases separately. Search time is measured separately from preprocessing; preprocessing is nevertheless included in the theoretical total-cost discussion.

Task 1 – Algorithm design

Approach A: Iterative Linear Search

Design strategy: sequential scan; works on unsorted records and requires no preprocessing. Compare the target with each key from the beginning until a match is found or the list ends.

LINEAR-SEARCH(A, target)

for i = 0 to length(A)-1

if A[i] == target: return i return NOT_FOUND

Selected because it is simple, supports unsorted data and provides a baseline. It can be slow when the match is late or absent.

Approach B: Recursive Binary Search

Design strategy: divide and conquer; requires the records to be sorted by registration number. Compare the middle key and recursively search only the half that can contain the target.

RECURSIVE-BINARY-SEARCH(A, target, low, high) if low > high: return NOT_FOUND

mid = floor((low + high)/2)

if A[mid] == target: return mid

if target < A[mid]: return RECURSIVE-BINARY-SEARCH(A,target,low,mid-1) else: return RECURSIVE-BINARY-SEARCH(A,target,mid+1,high)

Selected because halving the search interval gives logarithmic query work. Sorting is a preprocessing requirement and must be counted in the overall cost.

Approach C: Hash Table Search

Design strategy: hashing/indexing; build a dictionary mapping each registration number to its record position. A query hashes the key and looks up its entry. Expected lookup time is constant under a suitable hash distribution and controlled load factor; collisions can make worst-case lookup linear.

BUILD-HASH-TABLE(records)

H = empty hash table

for each record at index i: H[record.registration_number] = i return H

HASH-SEARCH(H, target)

if target is in H: return H[target] return NOT_FOUND

Selected because the challenge has many searches but only one daily update batch. The index can be built or updated when records are added, then reused for many queries.